

MASTER_ACTION_PLAN

HelixCode Zero-Bluff Master Action Plan

Version: 1.0.0 **Date:** 2026-04-30 **Status:** Comprehensive plan based on ALL bluff-proofing materials **Timeline:** 14 Weeks (7 Phases) **Current State:** 40% bluff / 60% real (by feature count) **Target State:** 100% real, zero-bluff, exceeds Tier 1 CLI agents

Executive Summary

This plan transforms HelixCode from its current state (with verified critical bluff areas) into a **fully complete, zero-bluff project** that exceeds all Tier 1 CLI agent capabilities.

Critical Bluffs Identified (MUST FIX):

- BLUFF-001:** LLM Generation is Completely Simulated (`cmd/cli/main.go:190-214`)
- BLUFF-002:** Model Listing is Hardcoded (`cmd/cli/main.go:101-128`)
- BLUFF-003:** Command Execution is Simulated (`cmd/cli/main.go:237-250`)
- BLUFF-004:** Worker Pool Statistics May Be Simulated (verification required)
- BLUFF-005:** Minimal Go Dependencies vs Advertised Features (`go.mod` only 3 deps)
- BLUFF-006:** Notification System - Unknown Real Implementation
- BLUFF-007:** Database Schema Advertised but Not Verified
- BLUFF-008:** Docker Compose References Non-Existent Files
- BLUFF-009:** Terminal UI Application Build Target

Verified Real Implementations:

- REAL-001:** Authentication System (`internal/auth/auth.go` - 470 lines, production-ready)
 - REAL-002:** CLI Structure and Flag Parsing (partial, 15+ flags)
 - REAL-003:** Dockerfile Multi-Stage Build (partial)
-

Anti-Bluff Mandate (CONST-035)

Every task in this plan **MUST** adhere to CONST-035: End-User Usability Mandate:

A test or Challenge that **PASSES** is a **CLAIM** that the tested behavior **works for the end user**.

Every **PASS** result **MUST** guarantee: - **Quality** - Feature behaves correctly under real user inputs, edge cases, concurrency - **Completion** - Feature is wired end-to-end with no stub/placeholder gaps - **Full Usability** - A user following documentation **SUCCEEDS**

Bluff Taxonomy (Forbidden Patterns):

- Wrapper bluff** - Assertions **PASS** but wrapper's exit-code logic is buggy

- **Contract bluff** - System advertises capability but rejects it in dispatch
 - **Structural bluff** - File exists but doesn't contain working code
 - **Comment bluff** - Comment promises behavior code doesn't have
 - **Skip bluff** - `t.Skip("not running yet")` without `SKIP-OK: #<ticket>` marker
-

Phase 0: Foundation Repair (Week 1)

P0-001: Fix go.mod - Add All Advertised Dependencies

File: `helix_code/go.mod` (replace root `go.mod`) **Current State:** Only 3 dependencies (`uuid`, `errors`, `yaml`) **Target State:** Full dependency manifest with 20+ dependencies

Sub-Tasks:

1. **P0-001-1:** Replace root `go.mod` with complete dependency list

- Module: `dev.helix.code`
- Go version: `go 1.24.0`
- Toolchain: `go1.24.9`

2. **P0-001-2:** Add HTTP Framework

- `github.com/gin-gonic/gin v1.11.0`

3. **P0-001-3:** Add Authentication dependencies

- `github.com/golang-jwt/jwt/v4 v4.5.2`
- `golang.org/x/crypto v0.36.0`

4. **P0-001-4:** Add Database dependencies

- `github.com/jackc/pgx/v5 v5.7.4`

5. **P0-001-5:** Add Redis dependencies

- `github.com/redis/go-redis/v9 v9.7.3`

6. **P0-001-6:** Add Configuration dependencies

- `github.com/spf13/viper v1.21.0`

7. **P0-001-7:** Add CLI dependencies

- `github.com/spf13/cobra v1.8.0`

8. **P0-001-8:** Add Testing dependencies

- `github.com/stretchr/testify v1.11.1`

9. **P0-001-9:** Add UI dependencies

- `github.com/rivo/tview v0.42.0`

- `fyne.io/fyne/v2 v2.7.0`

10. **P0-001-10:** Add Browser Automation

- `github.com/chromedp/chromedp v0.14.2`

11. **P0-001-11:** Add Web Scraping

- `github.com/PuerkitoBio/goquery v1.10.3`

12. **P0-001-12:** Add Tree-sitter

- `github.com/smacker/go-tree-sitter v0.0.0-20240625050157-a31a98a7c127`

13. **P0-001-13:** Add Observability

- `github.com/prometheus/client_golang v1.22.0`
- `github.com/sirupsen/logrus v1.9.3`

14. **P0-001-14:** Add SSE for MCP

- `github.com/r3labs/sse/v2 v2.10.0`

15. **P0-001-15:** Run `go mod tidy` and verify no errors

16. **P0-001-16:** Verify `go build ./...` compiles all packages

17. **P0-001-17:** Verify `go list -m all` shows all dependencies resolved

18. **P0-001-18:** Create unit test that verifies all imports resolve

19. **P0-001-19:** Create Challenge `go_mod_complete_challenge.sh`

- Challenge must verify all advertised deps are present
- Challenge must fail if deps missing
- Anti-bluff: `grep` for `gin`, `pgx`, `viper`, `cobra` in `go.mod`

Anti-Bluff Verification: - [] `go mod tidy` completes without errors - [] `go build ./...` compiles all packages - [] `go list -m all` shows all dependencies resolved - [] Challenge `go_mod_complete_challenge.sh` PASSES - [] NO “simulated” or “for now” in production code

P0-002: Create Missing `docker-entrypoint.sh`

File: `docker/docker-entrypoint.sh` (new file) **Current State:** Referenced in Dockerfile but doesn't exist

Sub-Tasks:

1. **P0-002-1:** Create `docker/docker-entrypoint.sh` with proper shebang and `set -e`

2. **P0-002-2:** Add validation for required environment variables

- `HELIX_AUTH_JWT_SECRET` (REQUIRED)
- `HELIX_DATABASE_PASSWORD` (WARNING if missing)

3. **P0-002-3:** Add wait-for-PostgreSQL logic

- Use `pg_isready` in loop until ready
- Timeout after 60 seconds

4. **P0-002-4:** Add wait-for-Redis logic

- Use `redis-cli ping` in loop until PONG
- Timeout after 60 seconds

5. **P0-002-5:** Add database migration execution

- Check for `./scripts/migrate.sh`
- Execute if exists

6. **P0-002-6:** Add service type selection

- `HELIX_SERVICE_TYPE` environment variable
- Support: `server`, `cli`, `worker`
- Use `exec` for proper signal handling

7. **P0-002-7:** Make script executable

- `chmod +x docker/docker-entrypoint.sh`

8. **P0-002-8:** Update Dockerfile to copy entrypoint to `/usr/local/bin/`

9. **P0-002-9:** Test: `docker build -t helixcode:test .`

10. **P0-002-10:** Create Challenge `docker_entrypoint_challenge.sh`

- Verify container builds without “file not found” errors
- Verify container actually waits for dependencies
- Verify health endpoint responds after start
- Anti-bluff: Actually curl health endpoint, verify deep checks

Anti-Bluff Verification: - [] Container builds successfully: `docker build -t helixcode:test .` - [] Container starts without “file not found” errors - [] Container actually waits for dependencies (not just echoes) - [] Health endpoint responds after container starts - [] Challenge `docker_entrypoint_challenge.sh` PASSES

P0-003: Add Root-Level Governance Files

Files: - `CONSTITUTION.md` (new at root) - `CLAUDE.md` (new at root) - `AGENTS.md` (update existing)

Sub-Tasks:

1. **P0-003-1:** Create CONSTITUTION.md at root
 - Include all 36 rules from HELIXCODE_CONSTITUTION.md
 - CONST-001: No CI/CD Pipelines
 - CONST-002: No Mocks in Production
 - CONST-003: No HTTPS for Git
 - CONST-004: No Manual Container Commands
 - CONST-005: 100% Real Data for Non-Unit Tests
 - CONST-006: Challenge Coverage
 - CONST-007: Health & Observability
 - CONST-008: Documentation & Quality
 - CONST-009: Validation Before Release
 - CONST-010: Comprehensive Verification
 - CONST-011: Resource Limits for Tests & Challenges
 - CONST-012: Bugfix Documentation
 - CONST-013: Real Infrastructure for All Non-Unit Tests
 - CONST-014: Reproduction-Before-Fix
 - CONST-015: Concurrent-Safe Containers
 - CONST-016: Definition of Done
 - CONST-017: Zero-Bluff Testing (CONST-035)
 - CONST-018: Host Power Management Hard Ban
 - CONST-019: Container Up ≠ Healthy
 - CONST-020: Provider Fallback Chain Reality
 - CONST-021: No Mocks Above Unit Build Target
 - CONST-022: Submodule Governance Propagation
 - CONST-023: Docker Health Checks Mandatory
 - CONST-024: Version Pinning
 - CONST-025: Secret Management
 - CONST-026: Minimal Privilege Containers
 - CONST-027: Network Isolation
 - CONST-028: Backup Before Destructive Operations
 - CONST-029: Input Validation at All Boundaries
 - CONST-030: Graceful Degradation
 - CONST-031: Audit Trail
 - CONST-032: Emergency Stop
 - CONST-033: Data Integrity
 - CONST-034: API Stability
 - CONST-035: End-User Usability Mandate (strengthened 2026-04-29)
 - CONST-036: Propagation to Submodules
2. **P0-003-2:** Create CLAUDE.md at root (491 lines)
 - Section 1: Agent Identity & Purpose
 - Section 2: Universal Mandatory Rules (10 rules)
 - Section 3: HelixCode-Specific Architecture
 - Section 4: Code Patterns for Agents
 - Section 5: Anti-Bluff Checklist for Every Task
 - Section 6: Common Anti-Patterns to Avoid
 - Section 7: Working with Submodules
 - Section 8: Emergency Procedures
 - Section 9: Reference Commands
 - Section 10: Contact & Escalation
3. **P0-003-3:** Update AGENTS.md at root
 - Combine existing AGENTS.md with HELIXCODE_AGENTS.md content
 - Add CONST-035 section with bluff taxonomy

- Add verified bluff areas (BLUFF-001 through BLUFF-009)
- Add verified real implementations (REAL-001 through REAL-003)
- Add anti-bluff testing rules
- 4. **P0-003-4:** Propagate to ALL submodules
 - For each submodule in `.gitmodules` (80+ submodules):
 - Option A: Create symlink to parent governance files
 - Option B: Copy governance files to submodule root
 - Option C: Add reference comment in submodule README
 - Verify: Every submodule has `Constitution.md`, `CLAUDE.md`, `AGENTS.md`
- 5. **P0-003-5:** Verify no forbidden patterns in production code
 - `grep -r "simulated" internal/ cmd/ applications/`
 - `grep -r "for now" internal/ cmd/ applications/`
 - `grep -r "TODO implement" internal/ cmd/ applications/`
 - `grep -r "placeholder" internal/ cmd/ applications/`
 - MUST return NOTHING
- 6. **P0-003-6:** Create Challenge `constitution_validation_challenge.sh`
 - Verify `CONSTITUTION.md` exists at root
 - Verify `CONST-035` is present
 - Verify all 36 rules are present
 - Verify `CLAUDE.md` exists at root (490+ lines)
 - Verify `AGENTS.md` exists at root (390+ lines)
 - Verify all submodules have governance files
 - Anti-bluff: Actually read files, not just check existence
- 7. **P0-003-7:** Create Challenge `no_simulation_challenge.sh`
 - Scan all production code for simulation markers
 - Must return 0 findings
 - Anti-bluff: Actually read file contents

Anti-Bluff Verification: - [] `CONSTITUTION.md` exists at root (330+ lines) - [] `CLAUDE.md` exists at root (490+ lines) - [] `AGENTS.md` exists at root (390+ lines) - [] All 80+ submodules have governance files - [] `grep` for “simulated” returns NOTHING in production code - [] Challenge `constitution_validation_challenge.sh` PASSES - [] Challenge `no_simulation_challenge.sh` PASSES

P0-004: Create Anti-Bluff Testing Framework

File: `tests/anti_bluff_framework.go` (new)

Sub-Tasks:

1. **P0-004-1:** Create `AntiBluffVerifier` struct
 - Wrap testing.T
 - Provide assertion methods that check REAL behavior
2. **P0-004-2:** Implement `AssertRealHTTPCall`
 - Check response does NOT contain “simulated”
 - Check response does NOT contain “This is a simulated”
 - Verify actual content is meaningful
3. **P0-004-3:** Implement `AssertRealFileChange`
 - Verify file was actually modified
 - Check expected content exists in file
 - Not just that function was called
4. **P0-004-4:** Implement `AssertRealProcessExecution`
 - Verify actual system calls were made

- Check process actually ran
- Verify output is real
- 5. **P0-004-5:** Create Challenge wrapper template
 - Use robust failure tracking (grep “|FAILED|” pattern)
 - NOT just exit codes
 - Continue running after failures
 - Generate JSON report
- 6. **P0-004-6:** Implement `run_all_challenges.sh` master runner
 - Run ALL challenges
 - Use robust failure detection
 - Generate JSON report with per-challenge results
 - Fail if ANY challenge fails
 - Respect resource limits: `GOMAXPROCS=2, nice -n 19`
- 7. **P0-004-7:** Create minimum 10 challenges for P0 features
 - `go_mod_complete_challenge.sh`
 - `docker_entrpoint_challenge.sh`
 - `constitution_validation_challenge.sh`
 - `no_simulation_challenge.sh`
 - `anti_bluff_framework_challenge.sh`
 - etc.

Anti-Bluff Verification: - [] Anti-bluff framework code exists - []
`run_all_challenges.sh` runs without buggy wrapper logic - [] All P0 challenges PASS - []
 [] Wrapper correctly propagates failures (grep “|FAILED|” pattern)

Phase 1: Core LLM Integration (Weeks 2-3)

P1-001: Implement Real LLM Provider Interface

File: `internal/llm/provider.go` (verify/update)

Sub-Tasks:

1. **P1-001-1:** Define complete Provider interface

```
type Provider interface {
    Generate(ctx context.Context, req *GenerateRequest)
        (*GenerateResponse, error)
    GenerateStream(ctx context.Context, req
        *GenerateRequest) (<-chan *GenerateChunk, error)
    GetCapabilities() *Capabilities
    GetModels() ([]Model, error) // MUST return real models
    ValidateConfig(config map[string]interface{}) error
    HealthCheck(ctx context.Context) error // NEW: for anti-
        bluff
}
```

2. **P1-001-2:** Define `GenerateRequest` struct

- Prompt, Model, MaxTokens, Temperature
- Stream flag, Context, etc.

3. **P1-001-3:** Define GenerateResponse struct

- Text, Model, Provider
- Completed flag, TokensUsed, etc.

4. **P1-001-4:** Define Model struct

- ID, Name, Provider
- ContextSize, Capabilities

5. **P1-001-5:** Create unit tests for interface compliance

6. **P1-001-6:** Create Challenge provider_interface_challenge.sh

- Verify interface is implemented
 - Verify all methods are present
 - Anti-bluff: Actually call methods, not just check existence
-

P1-002: Implement Ollama Provider (Local - Priority 1)

File: internal/llm/providers/ollama/ollama.go (new)

Sub-Tasks:

1. **P1-002-1:** Create OllamaProvider struct

- baseURL, httpClient
- Default URL: http://localhost:11434

2. **P1-002-2:** Implement Generate method

- ANTI-BLUFF: MUST make REAL HTTP call to Ollama API
- POST to /api/generate with proper JSON payload
- Parse response, return actual generated text
- NO simulation. NO echoing prompt. NO canned responses.

3. **P1-002-3:** Implement GenerateStream method

- Stream responses via SSE from Ollama
- Return channel of chunks
- Actually stream, not simulate

4. **P1-002-4:** Implement GetModels method

- ANTI-BLUFF: Query Ollama's /api/tags endpoint
- Return REAL available models, not hardcoded list
- Parse response, extract model names

5. **P1-002-5:** Implement HealthCheck method

- ANTI-BLUFF: Real health check, not just "return nil"
- Make HTTP request to /api/tags
- Verify 200 response

6. **P1-002-6:** Implement GetCapabilities method

- Return actual capabilities
- Context size, streaming support, etc.

7. **P1-002-7:** Add unit tests with mocks (allowed at unit level)

8. **P1-002-8:** Add integration test with REAL Ollama

- Skip if Ollama not available
- Make real generation request
- Verify response is not simulated
- Anti-bluff: Check response does NOT contain “simulated”

9. **P1-002-9:** Create Challenge `ollama_generation_challenge.sh`

- Start Ollama if not running
- Make generation request via API
- Verify response contains actual AI-generated text
- Verify response does NOT contain “simulated”
- Verify response answers the question (not just echoes)
- Anti-bluff: Check response has reasonable length (>50 chars)

10. **P1-002-10:** Create Challenge `ollama_models_challenge.sh`

- Query Ollama for available models
- Verify dynamic list (not hardcoded)
- Verify models actually exist in Ollama

Anti-Bluff Verification: - [] `./bin/cli --prompt "What is 2+2?"` returns REAL AI-generated answer - [] Response does NOT contain “simulated” or “This is a simulated” - [] Response actually answers: “4” or “four” - [] `./bin/cli --list-models` returns dynamic models from Ollama - [] Integration test passes with real Ollama - [] Challenge `ollama_generation_challenge.sh` PASSES - [] Challenge `ollama_models_challenge.sh` PASSES

P1-003: Implement OpenAI Provider (Cloud - Priority 2)

File: `internal/llm/providers/openai/openai.go` (new)

Sub-Tasks:

1. **P1-003-1:** Create OpenAIProvider struct
 - `baseUrl`: `https://api.openai.com/v1`
 - `apiKey` from config
 - `httpClient` with timeout
2. **P1-003-2:** Implement Generate method
 - REAL HTTP POST to `/v1/chat/completions`
 - Proper authentication header: `Bearer <api-key>`
 - Parse response, return actual generated text
3. **P1-003-3:** Implement GenerateStream method
 - Stream via SSE
 - Return channel of chunks

4. **P1-003-4:** Implement GetModels method
 - Query `/v1/models` endpoint
 - Return real available models
 5. **P1-003-5:** Implement HealthCheck method
 - Make test request
 - Verify API is accessible
 6. **P1-003-6:** Add integration test with REAL OpenAI API
 - Skip if `OPENAI_API_KEY` not set
 - Make real generation request
 - Verify response is not simulated
 7. **P1-003-7:** Create Challenge `openai_generation_challenge.sh`
 - Requires `OPENAI_API_KEY`
 - Verify real generation
 - Anti-bluff checks
-

P1-004: Implement Anthropic Claude Provider (Cloud - Priority 3)

File: `internal/llm/providers/anthropic/anthropic.go` (new)

Sub-Tasks:

1. **P1-004-1:** Create AnthropicProvider struct
 - `baseURL`: `https://api.anthropic.com/v1`
 - `apiKey`, `apiVersion` from config
 2. **P1-004-2:** Implement Generate method
 - REAL HTTP POST to `/v1/messages`
 - Proper authentication: `x-api-key` header
 - Support Claude 3.5 Sonnet, Opus, Haiku
 - Parse response, return actual text
 3. **P1-004-3:** Implement extended thinking mode
 - Support 200K context
 - Support 50K output tokens
 4. **P1-004-4:** Implement GetModels method
 - Return advertised Claude models
 5. **P1-004-5:** Create Challenge `anthropic_generation_challenge.sh`
 - Requires `ANTHROPIC_API_KEY`
 - Verify real generation with Claude
-

P1-005: Implement 12 More Providers

Files: `internal/llm/providers/*/provider.go` (new for each)

Providers to Implement:

1. **Gemini** (`internal/llm/providers/gemini/`)
 - 2M token context (largest available)
 - Multimodal, function calling
2. **xAI/Grok** (`internal/llm/providers/xai/`)
 - `grok-3-fast-beta`, `grok-3-mini-fast-beta`
 - Free tier available

3. **OpenRouter** (internal/llm/providers/openrouter/)
 - Free models from various providers
 - deepseek-r1-free, meta-llama/llama-3.2-3b-instruct:free
4. **GitHub Copilot** (internal/llm/providers/github/)
 - gpt-4o, claude-3.5-sonnet, o1
 - Free with GitHub subscription
5. **Qwen** (internal/llm/providers/qwen/)
 - 2,000 requests/day free tier
 - OAuth2 authentication
6. **Llama.cpp** (internal/llm/providers/llamacpp/)
 - Local server via HTTP
 - /completion endpoint
7. **vLLM** (internal/llm/providers/vllm/)
 - Local server via HTTP
 - /v1/completions endpoint
8. **Azure Bedrock** (internal/llm/providers/azure/)
 - Azure's enterprise OpenAI models
9. **AWS Bedrock** (internal/llm/providers/bedrock/)
 - AWS SDK integration
 - Claude, Titan, Jurassic models
10. **VertexAI** (internal/llm/providers/vertexai/)
 - Google's enterprise models
11. **Groq** (internal/llm/providers/groq/)
 - Ultra-fast inference
 - Llama and Mixtral models
12. **KoboldAI** (internal/llm/providers/koboldai/)
 - Local server via HTTP
 - /api/v1/generate endpoint

For EACH provider, implement ALL: - Generate method (REAL HTTP call) - GenerateStream method (REAL streaming) - GetModels method (REAL discovery) - HealthCheck method (REAL probe) - GetCapabilities method - Unit tests (mocks allowed) - Integration test (real API, skip if unavailable) - Challenge script (anti-bluff verification)

P1-006: Implement Provider Manager with Fallback

File: internal/llm/manager.go (verify/fix)

Sub-Tasks:

1. **P1-006-1:** Create ProviderManager struct
 - providers map[string]Provider
 - fallbackChain []string (ordered list)
 - circuitBreakers map[string]*CircuitBreaker
2. **P1-006-2:** Implement RegisterProvider method
 - Add provider to map
 - Initialize circuit breaker

3. **P1-006-3:** Implement Generate method with fallback

- ANTI-BLUFF: Must try REAL providers
- Iterate through fallbackChain
- Skip open/circuit-broken providers
- Call provider.Generate()
- Record success/failure to circuit breaker
- Return first successful response

4. **P1-006-4:** Implement GenerateStream with fallback

- Similar to Generate but with streaming

5. **P1-006-5:** Implement model discovery across providers

- Query all providers for models
- Merge and deduplicate
- Cache with TTL

6. **P1-006-6:** Add unit tests

7. **P1-006-7:** Add integration test

- Test fallback chain with real providers
- Test circuit breaker opens on failures

8. **P1-006-8:** Create Challenge provider_fallback_challenge.sh

- Test fallback from failing provider to working provider
- MUST use real failures (not simulated)
- Verify circuit breaker opens
- Anti-bluff: Actually cause a provider to fail

P1-007: Implement Circuit Breaker Pattern

File: internal/llm/circuit_breaker.go (new - port from HelixAgent)

Sub-Tasks:

1. **P1-007-1:** Create CircuitBreaker struct

- state (Closed, Open, HalfOpen)
- failureCount, successCount
- threshold, timeout
- lastFailure time
- sync.RWMutex for thread safety

2. **P1-007-2:** Implement State method

- Return current circuit state

3. **P1-007-3:** Implement RecordSuccess method

- Reset failure count if in HalfOpen
- Transition to Closed if success threshold met

4. **P1-007-4:** Implement RecordFailure method

- Increment failure count
- Transition to Open if threshold exceeded
- Record lastFailure time

5. **P1-007-5:** Implement CanAttempt method
 - Return true if Closed
 - Return true if HalfOpen and timeout expired
 - Return false if Open
 6. **P1-007-6:** Add unit tests
 - Test state transitions
 - Test threshold behavior
 7. **P1-007-7:** Create Challenge `circuit_breaker_challenge.sh`
 - Cause real provider failures
 - Verify circuit opens
 - Verify requests are skipped when open
 - Anti-bluff: Actually cause failures
-

P1-008: Fix CLI `handleGenerate` to Use Real Providers

File: `cmd/cli/main.go` lines 190-214 (CRITICAL FIX)

Sub-Tasks:

1. **P1-008-1:** Replace simulated generation with real provider call
 - Remove “For now, simulate generation” comment
 - Remove simulated response code
 - ANTI-BLUFF: MUST use real LLM provider
2. **P1-008-2:** Initialize provider from config or default to local
 - Check config for `default_provider`
 - Fall back to Ollama if local
3. **P1-008-3:** Create `GenerateRequest` from CLI flags
 - Prompt, Model, MaxTokens, Temperature
4. **P1-008-4:** Call `provider.Generate` (or `GenerateStream` for streaming)
 - Handle errors properly
 - Print actual response text
5. **P1-008-5:** Fix streaming mode
 - Actually stream from provider
 - Print chunks as they arrive
 - NOT simulate with `time.Sleep`
6. **P1-008-6:** Add unit tests
7. **P1-008-7:** Create Challenge `cli_generation_challenge.sh`
 - Run `./bin/cli --prompt "What is capital of France?"`
 - Verify response contains “Paris”
 - Verify response does NOT contain “simulated”
 - Verify response is not just echoing prompt
 - Anti-bluff: Check response quality

8. **P1-008-8:** Verify no “simulated” in production code

- `grep -n "simulated" cmd/cli/main.go`
- MUST return nothing

Anti-Bluff Verification for Phase 1: - [] `go test ./internal/llm/... passes` - [] Integration test calls real Ollama instance (or mock server in test) - [] `./bin/cli --prompt "What is 2+2?"` returns actual AI-generated text - [] `./bin/cli --list-models` returns dynamic models from running provider - [] Health check verifies provider is actually responding - [] Challenge script generates a real project using LLM calls - [] NO “simulated” or “for now” comments in production LLM code - [] All 15+ providers have real implementations - [] Circuit breakers work for all providers - [] Fallback chain tested with real failures

Phase 2: Tools & Editor (Weeks 4-5)

P2-001: Implement Real Filesystem Tools

Files: `internal/tools/filesystem/*.go`

Sub-Tasks:

1. **P2-001-1:** Implement `fs_read` tool

- Actually read file from disk using `os.ReadFile`
- Path validation to prevent directory traversal
- Return file contents

2. **P2-001-2:** Implement `fs_write` tool

- Actually write file to disk using `os.WriteFile`
- Atomic operation (write to temp, rename)
- Create backup before overwrite

3. **P2-001-3:** Implement `fs_edit` tool

- Edit file with backup
- Support various edit formats
- Verify file actually modified

4. **P2-001-4:** Implement `glob` tool

- Pattern matching using `filepath.Glob`
- Return actual matching files

5. **P2-001-5:** Implement `grep` tool

- Content search using `regexp`
- Return actual matching lines

6. **P2-001-6:** Add path validation for all tools

- Prevent `../` traversal
- Restrict to workspace directory

7. **P2-001-7:** Add unit tests with mocks
 8. **P2-001-8:** Add integration tests with real temp files
 - ANTI-BLUFF: Use real temp files on disk
 - Verify file actually created/read/written
 9. **P2-001-9:** Create Challenge `filesystem_tools_challenge.sh`
 - Write a file, read it back
 - Verify content matches
 - Edit file, verify change
 - Glob for files, verify results
 - Grep for content, verify matches
 - Anti-bluff: Actually check file contents on disk
-

P2-002: Implement Real Shell Tool with Sandboxing

File: `internal/tools/shell/shell.go`

Sub-Tasks:

1. **P2-002-1:** Implement `ShellTool.Execute`
 - ANTI-BLUFF: MUST execute real shell commands
 - Use `os/exec.CommandContext`
 - Set working directory
2. **P2-002-2:** Add command validation against blocklist
 - Prevent dangerous commands (`rm -rf /`, etc.)
 - Configurable blocklist
3. **P2-002-3:** Add timeout and resource limits
 - `Context.WithTimeout`
 - Nice values, `ionice`
4. **P2-002-4:** Capture `stdout/stderr`
 - Return actual output
 - Return actual exit code
5. **P2-002-5:** Implement `shell_background` tool
 - Start long-running process
 - Return process ID
6. **P2-002-6:** Implement `shell_output` tool
 - Quick output capture

7. **P2-002-7:** Implement `shell_kill` tool
 - Kill background process by ID
 8. **P2-002-8:** Add unit tests
 9. **P2-002-9:** Add integration test with real commands
 - ANTI-BLUFF: Actually run `echo hello`
 - Verify output is “hello”
 - Not just simulation
 10. **P2-002-10:** Create Challenge `shell_execution_challenge.sh`
 - Run `echo BLUFF_TEST_12345`
 - Verify output contains “BLUFF_TEST_12345”
 - Run `ls` and verify actual file listing
 - Anti-bluff: Check real output, not canned
-

P2-003: Implement Git Tool

File: `internal/tools/git/git.go`

Sub-Tasks:

1. **P2-003-1:** Implement `git_status` tool
 - Run `git status` in repo
 - Return actual status output
 2. **P2-003-2:** Implement `git_diff` tool
 - Run `git diff`
 - Return actual diff
 3. **P2-003-3:** Implement `git_add` tool
 - Run `git add` for files
 4. **P2-003-4:** Implement `git_commit` tool
 - Generate smart commit message
 - Run `git commit`
 5. **P2-003-5:** Implement `git_branch` tool
 - List or create branches
 6. **P2-003-6:** Implement `git_blame` tool
 - Attribution for code lines
 7. **P2-003-7:** Add integration tests with real git repo
 - ANTI-BLUFF: Actually run git commands
 - Verify real git operations
 8. **P2-003-8:** Create Challenge `git_tools_challenge.sh`
 - Initialize git repo
 - Create a file, commit it
 - Check status, verify clean
 - Anti-bluff: Actually check git log
-

P2-004: Implement Browser Automation Tools

File: `internal/tools/browser/browser.go`

Sub-Tasks:

1. **P2-004-1:** Implement `browser_launch` tool
 - Use `chromedp` to launch browser
 - REAL browser automation, not simulation
 2. **P2-004-2:** Implement `browser_navigate` tool
 - Navigate to URL
 - Wait for page load
 3. **P2-004-3:** Implement `browser_screenshot` tool
 - Take actual screenshot
 - Save to file or return bytes
 4. **P2-004-4:** Implement `browser_click`, `browser_type`, etc.
 - Full browser interaction
 5. **P2-004-5:** Add integration tests with real browser
 - Launch, navigate, screenshot
 - ANTI-BLUFF: Verify screenshot file created
 6. **P2-004-6:** Create Challenge `browser_automation_challenge.sh`
 - Launch browser
 - Navigate to URL
 - Take screenshot
 - Verify screenshot exists and has content
 - Anti-bluff: Actually check image file
-

P2-005: Implement Editor with Multi-Format Support

File: `internal/editor/editor.go`

Sub-Tasks:

1. **P2-005-1:** Implement Diff Format editing
 - Parse unified diff
 - Apply to file
 - Best for GPT-4, Gemini Pro, DeepSeek Coder
2. **P2-005-2:** Implement Whole File replacement
 - Replace entire file content
 - Best for Claude, O1 models, Llama 3 8B
3. **P2-005-3:** Implement Search/Replace editing
 - Pattern-based with regex
 - Best for Claude, GPT-3.5, Mistral
4. **P2-005-4:** Implement Line-Based editing
 - Edit specific line ranges
 - Best for GPT-4, Claude, Gemini
5. **P2-005-5:** Add automatic format selection
 - Based on model capabilities
6. **P2-005-6:** Add thread-safe concurrent editing
 - Use `sync.RWMutex`
7. **P2-005-7:** Add built-in validation
 - Verify edit actually applied
8. **P2-005-8:** Add backup support
 - Create backup before edit
 - Restorable backups

9. **P2-005-9:** Create Challenge `editor_challenge.sh`

- Apply each edit format
 - Verify file actually changed
 - Check backup created
 - Anti-bluff: Read file after edit, verify content
-

P2-006: Implement Additional Tools

Files: `internal/tools/*/tool.go`

Tools to Implement:

1. **Web Tools** (`internal/tools/web/`)
 - `web_fetch`: Fetch URL content (REAL HTTP GET)
 - `web_search`: Search with rate limiting and caching
2. **MultiEdit Tool** (`internal/tools/multiedit/`)
 - Transactional multi-file editing
 - All-or-nothing rollback
3. **Mapping Tool** (`internal/tools/mapping/`)
 - Codebase analysis with treesitter
 - Parse code structure
4. **Voice Tool** (`internal/tools/voice/`)
 - Voice input and transcription
5. **Confirmation Tool** (`internal/tools/confirmation/`)
 - User interaction with audit trails
6. **Notebook Tool** (`internal/tools/notebook/`)
 - Jupyter notebook integration

For EACH tool, implement ALL: - Real implementation (no simulation) - Schema validation - Execute method - Unit tests (mocks allowed) - Integration tests (real infrastructure) - Challenge script (anti-bluff)

Anti-Bluff Verification for Phase 2: - [] `fs_write` actually creates files - [] `shell` actually runs commands and returns real output - [] `git_status` returns real git status - [] Editor formats apply actual changes to files - [] Browser tools actually launch browser and take screenshots - [] Challenge: Edit a file and verify the change persists - [] Challenge: Run command and verify output is real - [] ALL 40+ tools have real implementations - [] ALL tools have challenges that verify real behavior

Phase 3: Worker & Distributed Computing (Weeks 6-7)

P3-001: Verify/Fix SSH Worker Implementation

Files: `internal/worker/ssh.go`, `internal/worker/pool.go`

Sub-Tasks:

1. **P3-001-1:** Read current implementation
 - Determine if real or simulated

- If simulated, replace with real
 - 2. **P3-001-2:** Implement real SSH connection management
 - Use `golang.org/x/crypto/ssh`
 - Real TCP connection to worker
 - 3. **P3-001-3:** Implement health checks
 - Real TCP/SSH probes
 - Periodic check every 30s
 - Update `last_heartbeat`
 - 4. **P3-001-4:** Implement task distribution
 - Assign tasks to real workers
 - Based on capability matching (CPU, GPU, memory)
 - 5. **P3-001-5:** Implement result collection
 - Gather actual execution results
 - Not just “Command completed successfully”
 - 6. **P3-001-6:** Implement worker auto-installation
 - Via SSH `exec`
 - Check if `helix` binary exists
 - If not, install via package or compile
 - 7. **P3-001-7:** Add unit tests
 - 8. **P3-001-8:** Add integration test with real SSH
 - Skip if no SSH available
 - Connect to real worker
 - Execute real command
 - Verify result
 - 9. **P3-001-9:** Create Challenge `ssh_worker_challenge.sh`
 - Add worker via SSH
 - Verify connection successful
 - Run task on worker
 - Verify task actually executed
 - Anti-bluff: Check real SSH connection
-

P3-002: Implement Task Checkpointing

File: `internal/task/checkpoint.go`

Sub-Tasks:

1. **P3-002-1:** Define Checkpoint struct
 - TaskID, Sequence, State (map), CreatedAt
 2. **P3-002-2:** Implement CreateCheckpoint
 - ANTI-BLUFF: Must persist to real database
 - Not just in-memory map
 - Use db.SaveCheckpoint()
 3. **P3-002-3:** Implement LoadCheckpoint
 - Load from database
 - Restore task state
 4. **P3-002-4:** Implement automatic checkpointing
 - Every 300s (configurable)
 - In background goroutine
 5. **P3-002-5:** Add database migration for checkpoints table
 6. **P3-002-6:** Create Challenge `checkpoint_challenge.sh`
 - Start long-running task
 - Verify checkpoint created in database
 - Kill task, restart
 - Verify state restored from checkpoint
 - Anti-bluff: Actually query database
-

P3-003: Implement Real Task Distribution

File: `internal/task/distributor.go`

Sub-Tasks:

1. **P3-003-1:** Implement task assignment
 - Assign to real workers based on capability
 - Not just round-robin simulation
2. **P3-003-2:** Implement progress monitoring
 - Track actual progress
 - Not just mark “running”
3. **P3-003-3:** Implement result collection
 - Gather from actual worker execution
 - Handle timeouts and failures
4. **P3-003-4:** Implement failover
 - On worker failure, retry on different worker
 - Respect `max_retries`
5. **P3-003-5:** Create Challenge `task_distribution_challenge.sh`
 - Submit task
 - Verify it runs on real worker

- Check result is actual output
 - Test failover by killing worker
 - Anti-bluff: Verify task actually executed
-

P3-004: Implement Worker Pool Statistics

File: `internal/worker/stats.go`

Sub-Tasks:

1. **P3-004-1:** Implement `GetWorkerStats`
 - Return REAL statistics
 - Not fabricated numbers
 - Query actual worker health
2. **P3-004-2:** Implement worker count by status
 - Healthy, unhealthy, offline
3. **P3-004-3:** Implement task queue statistics
 - Pending, running, completed, failed
4. **P3-004-4:** Create Challenge `worker_stats_challenge.sh`
 - Add real workers
 - Query stats
 - Verify numbers match reality
 - Anti-bluff: Actually count workers

Anti-Bluff Verification for Phase 3: - [] Worker pool connects to real SSH endpoints - [] Task execution happens on remote workers - [] Checkpoints are stored in PostgreSQL - [] Task failures trigger retry on different workers - [] Challenge: Distribute a build task across 2 workers - [] All worker operations are REAL, not simulated - [] All task operations persist to real database

Phase 4: Workflow & Session (Weeks 8-9)

P4-001: Implement Workflow Engine

File: `internal/workflow/engine.go`

Sub-Tasks:

1. **P4-001-1:** Define Workflow struct
 - ID, Name, Steps, Status
 - Dependencies between steps
2. **P4-001-2:** Define Step struct
 - ID, Type, Action, Dependencies
 - Config map
3. **P4-001-3:** Implement workflow execution
 - ANTI-BLUFF: Must execute real steps

- Resolve dependencies
- Execute with real action calls
- Not just mark complete

4. **P4-001-4:** Implement action registry

- Register action handlers
- Support: shell, git, llm, etc.

5. **P4-001-5:** Implement rollback on failure

- Undo completed steps
- Restore previous state

6. **P4-001-6:** Add unit tests

7. **P4-001-7:** Add integration test with real actions

- ANTI-BLUFF: Actually run workflow
- Verify steps executed

8. **P4-001-8:** Create Challenge `workflow_execution_challenge.sh`

- Define workflow with multiple steps
- Execute workflow
- Verify all steps ran
- Verify results are real
- Anti-bluff: Check actual artifacts created

P4-002: Implement Session Management with Redis

File: `internal/session/manager.go`

Sub-Tasks:

1. **P4-002-1:** Implement session creation
 - Store in REAL Redis (not memory)
 - Use provided Redis client
2. **P4-002-2:** Implement session retrieval
 - Fetch from Redis
 - Handle TTL and expiration
3. **P4-002-3:** Implement context storage
 - Mentions, search results, history
 - Store as JSON in Redis
4. **P4-002-4:** Implement multi-session tracking
 - List all sessions for user
 - Switch between sessions
5. **P4-002-5:** Add Redis connection check
 - Health check for Redis
6. **P4-002-6:** Create Challenge `session_persistence_challenge.sh`
 - Create session
 - Add context
 - Restart server (or simulate)
 - Verify session data persists

- Anti-bluff: Actually query Redis
-

P4-003: Implement Project Lifecycle

File: `internal/project/manager.go`

Sub-Tasks:

1. **P4-003-1:** Implement project creation
 - Store in real database
 - Not just in-memory
 2. **P4-003-2:** Implement lifecycle transitions
 - planning -> building -> testing -> deploying
 - Validate transitions
 3. **P4-003-3:** Implement task association
 - Link tasks to projects
 - Query project tasks
 4. **P4-003-4:** Implement artifact generation
 - Generate actual project files
 - Build outputs, test reports
 5. **P4-003-5:** Create Challenge `project_lifecycle_challenge.sh`
 - Create project
 - Transition through lifecycle
 - Verify state changes in database
 - Verify artifacts generated
 - Anti-bluff: Query database, check files
-

P4-004: Implement Database Migrations

Files: `internal/database/migrations/*.sql`

Sub-Tasks:

1. **P4-004-1:** Create migration for users table
 - ID, username, email, password_hash, etc.
2. **P4-004-2:** Create migration for sessions table
3. **P4-004-3:** Create migration for workers table
4. **P4-004-4:** Create migration for tasks table
 - With checkpoint_data JSONB
5. **P4-004-5:** Create migration for projects table
6. **P4-004-6:** Create migration for llm_providers table
7. **P4-004-7:** Create migration for notifications table

8. **P4-004-8:** Implement migration runner

- Apply migrations in order
- Track applied migrations

9. **P4-004-9:** Create Challenge `database_migration_challenge.sh`

- Run migrations
- Verify all 11 tables created
- Verify foreign keys work
- Anti-bluff: Query `information_schema`

Anti-Bluff Verification for Phase 4: - [] Workflow executes all steps in order - [] Session data persists across restarts (via Redis) - [] Project state changes are saved to database - [] Challenge: Create project, run workflow, verify artifacts - [] All database operations use real PostgreSQL - [] All session operations use real Redis - [] All 11 tables exist and are properly linked

Phase 5: MCP, Memory & Notifications (Weeks 10-11)

P5-001: Full MCP Protocol Implementation

File: `internal/mcp/server.go`

Sub-Tasks:

1. **P5-001-1:** Implement stdio transport
 - Read from stdin, write to stdout
 - JSON-RPC 2.0 protocol
 2. **P5-001-2:** Implement SSE transport
 - Server-Sent Events via HTTP
 - Real bidirectional communication
 3. **P5-001-3:** Implement tool registration
 - Register tools with MCP server
 - Execute tool via MCP protocol
 4. **P5-001-4:** Implement resource management
 - Expose resources via MCP
 5. **P5-001-5:** Create Challenge `mcp_protocol_challenge.sh`
 - Start MCP server (stdio or SSE)
 - Register tool
 - Execute tool via protocol
 - Verify real execution
 - Anti-bluff: Actually invoke tool
-

P5-002: Real Memory Provider Integration

Files: `internal/memory/providers/*.go`

Memory Providers to Implement (9 total):

1. **Mem0** - Advanced memory with embeddings
2. **Zep** - Long-term conversational memory

3. **Memonto** - Knowledge graph-based
4. **BaseAI** - Comprehensive platform
5. **ChromaDB** - Vector database
6. **FAISS** - Facebook AI's vector library
7. **Pinecone** - Managed vector database
8. **Qdrant** - Vector database
9. **Weaviate** - Knowledge graph with vector search

For EACH provider, implement: - Real API client (HTTP calls) - HealthCheck method - Store and retrieve operations - Integration test (skip if unavailable) - Challenge script

P5-003: Real Notification Dispatch

Files: `internal/notification/channels/*.go`

Channels to Implement (4 total):

1. **Slack** - Real webhook POST
2. **Discord** - Real webhook POST
3. **Email** - Real SMTP connection
4. **Telegram** - Real bot API call

For EACH channel, implement: - Send method with real HTTP/SMTP calls - Validate configuration - Integration test (skip if unavailable) - Challenge script

P5-004: Implement Health Endpoints

File: `internal/server/health.go`

Sub-Tasks:

1. **P5-004-1:** Implement `/health` endpoint
 - Deep checks (not just return 200)
 - Check database connection (SELECT 1)
 - Check Redis connection (PING)
 - Check provider availability (real generation request)
2. **P5-004-2:** Implement `/metrics` endpoint
 - Prometheus metrics
 - Request counts, durations, etc.
3. **P5-004-3:** Add health checks to Dockerfile
 - HEALTHCHECK instruction
 - Interval, timeout, retries
4. **P5-004-4:** Create Challenge `health_endpoint_challenge.sh`
 - Start server
 - Call `/health`
 - Verify deep checks passed
 - Verify metrics endpoint returns data
 - Anti-bluff: Actually check dependencies

Anti-Bluff Verification for Phase 5: - [] MCP server accepts real connections - [] Memory providers store and retrieve actual data - [] Notifications arrive at real endpoints - [] Challenge:

Send notification, verify receipt - [] All 9 memory providers have real implementations - [] All 4 notification channels work - [] Health endpoint performs deep checks

Phase 6: Testing & Challenges (Weeks 12-13)

P6-001: Achieve 100% Test Coverage

Sub-Tasks:

1. **P6-001-1:** Audit current test coverage
 - `go test -cover ./...`
 - Identify untested packages
 2. **P6-001-2:** Write unit tests for all packages
 - Target: 80%+ coverage per package
 - Use testify for assertions
 - Table-driven tests with `t.Run()`
 3. **P6-001-3:** Write contract tests for all providers
 - Test real API schemas
 - Use recorded responses (vcr-style) for CI
 - Skip (not fail) if API unavailable
 4. **P6-001-4:** Write component tests
 - Real subsystems wired together
 - Use testcontainers for PostgreSQL, Redis
 - Clean state between tests
 5. **P6-001-5:** Write integration tests
 - Full app with real dependencies
 - Start full docker-compose stack
 - Make real HTTP requests
 6. **P6-001-6:** Write security tests
 - SQL injection attempts
 - Path traversal tests
 - Command injection tests
 - JWT tampering tests
 7. **P6-001-7:** Write performance benchmarks
 - LLM generation latency
 - Concurrent task handling
 - Database query performance
 8. **P6-001-8:** Verify NO mocks above unit level
 - Create `no-mocks-above-unit` make target
 - Fail build if mocks found outside `*_test.go`
 9. **P6-001-9:** Verify resource limits
 - `GOMAXPROCS=2` for all tests
 - `nice -n 19` for all tests
 - `ionice -c 3` for all tests
-

P6-002: Create 100+ Challenges

Challenge Categories (per CONST-035):

1. **System Boot** (5 challenges)
 - full_system_boot_challenge.sh
 - docker_health_challenge.sh
 - etc.
2. **Constitution** (3 challenges)
 - constitution_validation_challenge.sh
 - no_simulation_challenge.sh
 - governance_propagation_challenge.sh
3. **LLM Providers** (15 challenges)
 - ollama_generation_challenge.sh
 - openai_generation_challenge.sh
 - anthropic_generation_challenge.sh
 - etc. (one per provider)
4. **Tools** (20 challenges)
 - shell_execution_challenge.sh
 - filesystem_tools_challenge.sh
 - git_tools_challenge.sh
 - browser_automation_challenge.sh
 - editor_challenge.sh
 - etc. (one per tool)
5. **Workflows** (10 challenges)
 - plan_build_test_challenge.sh
 - workflow_execution_challenge.sh
 - project_lifecycle_challenge.sh
6. **Security** (5 challenges)
 - auth_penetration_challenge.sh
 - sql_injection_challenge.sh
 - command_injection_challenge.sh
7. **Performance** (5 challenges)
 - concurrent_tasks_challenge.sh
 - llm_latency_challenge.sh
8. **Anti-Bluff** (10 challenges)
 - no_simulation_challenge.sh
 - provider_fallback_challenge.sh
 - circuit_breaker_challenge.sh
9. **Containers** (5 challenges)
 - docker_health_challenge.sh
 - docker_compose_challenge.sh
10. **Submodules** (10 challenges)
 - One per major submodule
 - Verify they actually work

For EACH challenge, implement: - Robust failure tracking (grep “|FAILED|” pattern) - Anti-bluff checks (verify REAL behavior) - JSON report generation - Skip properly if infrastructure unavailable (with SKIP-OK marker)

P6-003: Master Challenge Runner

File: tests/e2e/challenges/run_all_challenges.sh

Sub-Tasks:

1. **P6-003-1:** Run ALL challenges
 - Loop through challenges directory
 - Execute each challenge script
 2. **P6-003-2:** Use robust failure detection
 - grep “|FAILED|” in log files
 - NOT just exit codes
 3. **P6-003-3:** Generate JSON report
 - Per-challenge results
 - Passed, failed, skipped counts
 - Log file paths
 4. **P6-003-4:** Fail if ANY challenge fails
 - No silent swallowing
 5. **P6-003-5:** Respect resource limits
 - GOMAXPROCS=2
 - nice -n 19
 - ionice -c 3
 6. **P6-003-6:** Create sample runner
 - Run 10% of challenges
 - For quick smoke tests
-

P6-004: Anti-Bluff Testing Framework Enhancement

File: tests/anti_bluff_framework.go (enhance)

Sub-Tasks:

1. **P6-004-1:** Add more verification methods
 - AssertRealDatabaseWrite
 - AssertRealHTTPResponse
 - AssertRealFileModification
2. **P6-004-2:** Add concurrency testing
 - Verify thread-safety
 - Run with goroutines
3. **P6-004-3:** Add failure injection testing
 - Simulate provider failures
 - Verify circuit breakers
4. **P6-004-4:** Create Challenge anti_bluff_framework_challenge.sh
 - Verify framework works
 - Run sample verifications

Anti-Bluff Verification for Phase 6: - [] Unit test coverage: 80%+ per package - []
run_all_challenges.sh fails when BLUFF-001 is present - [] Test framework detects
simulated responses - [] Wrapper correctly reports failures - [] Challenge validates actual end-
user workflow - [] 100% of P0 features have challenges - [] 80% of P1 features have challenges
- [] 100+ total challenges created - [] ALL challenges PASS with real behavior

Phase 7: Documentation & Deployment (Week 14)

P7-001: Update All Documentation

Sub-Tasks:

1. **P7-001-1:** Update README.md
 - Reflect actual implemented features (not aspirational)
 - Include working code examples
 - Verify examples work when copy-pasted
 2. **P7-001-2:** Update docs/ARCHITECTURE_AND_DIAGRAMS.md
 - Mark all components as REAL or BLUFF (now all REAL)
 - Update diagrams
 3. **P7-001-3:** Update API documentation
 - List all actual endpoints
 - Include real request/response examples
 4. **P7-001-4:** Create docs/DEPLOYMENT.md
 - Docker Compose with real health checks
 - Kubernetes manifests
 - Environment variable reference
 - Troubleshooting with real diagnostic commands
 5. **P7-001-5:** Create docs/DEVELOPER_GUIDE.md
 - Setup instructions
 - Build instructions
 - Test instructions
 - How to add new features
 6. **P7-001-6:** Verify no “simulated” in documentation examples
 7. **P7-001-7:** Create Challenge documentation_challenge.sh
 - Copy-paste each README example
 - Verify it actually works
 - Anti-bluff: Actually execute examples
-

P7-002: Create Kubernetes Manifests

Directory: k8s/

Sub-Tasks:

1. **P7-002-1:** Create namespace.yaml

2. **P7-002-2:** Create secrets.yaml
 - JWT secret, database password, redis password
 3. **P7-002-3:** Create configmap.yaml
 - Application configuration
 4. **P7-002-4:** Create deployment.yaml
 - HelixCode server deployment
 - 3+ replicas with HPA
 5. **P7-002-5:** Create service.yaml
 - LoadBalancer or ClusterIP
 6. **P7-002-6:** Create postgres-statefulset.yaml
 7. **P7-002-7:** Create redis-cluster.yaml
 8. **P7-002-8:** Create ingress.yaml
 - nginx-ingress with TLS
 9. **P7-002-9:** Create monitoring manifests
 - Prometheus + Grafana
 10. **P7-002-10:** Create Challenge k8s_deployment_challenge.sh
 - Deploy to K8s cluster
 - Verify all pods running
 - Verify health checks pass
 - Anti-bluff: Actually curl health endpoint
-

P7-003: Submodule Governance Propagation

Sub-Tasks:

1. **P7-003-1:** For each submodule in `.gitmodules` (80+):
 - Check if Constitution.md exists
 - If not, create symlink or copy from parent
 - Verify CLAUDE.md exists
 - Verify AGENTS.md exists
2. **P7-003-2:** Add reference to parent governance
 - If submodule can't have files, add README note
 - Point to parent Constitution.md
3. **P7-003-3:** Verify anti-bluff testing requirements propagated
 - Each submodule should have its own challenges
 - Or reference parent's challenges
4. **P7-003-4:** Create Challenge submodule_governance_challenge.sh
 - Check all submodules have governance files
 - Verify they reference parent

- Anti-bluff: Actually read files
-

P7-004: Final Validation

Sub-Tasks:

1. **P7-004-1:** Run full test suite
 - `make test` (unit tests)
 - `make integration-test` (integration tests)
 - `make benchmark` (performance tests)
 - `make security-test` (security tests)
 2. **P7-004-2:** Run ALL challenges
 - `./tests/e2e/challenges/run_all_challenges.sh`
 - Verify ALL PASS
 3. **P7-004-3:** Build all targets
 - `make build` (Linux)
 - `make prod` (cross-platform)
 - `make mobile` (iOS + Android)
 - `make aurora-os`
 - `make harmony-os`
 4. **P7-004-4:** Docker Compose verification
 - `docker-compose up -d`
 - Verify all containers healthy
 - `docker-compose ps`
 5. **P7-004-5:** Documentation verification
 - Verify all README examples work
 - Copy-paste and execute
 6. **P7-004-6:** Create FINAL_VALIDATION_REPORT.md
 - Summary of all work done
 - List of all bluffs fixed
 - Test coverage numbers
 - Challenge results
 - Remaining work (if any)
-

P7-005: Success Criteria Checklist

HelixCode achieves zero-bluff status when ALL of the following are TRUE:

☐

`./bin/cli --prompt "What is 2+2?"` returns a real AI-generated answer (not simulated)

☐

`./bin/cli --list-models` returns dynamic models from running providers

☐

`./bin/cli --command "echo hello"` actually runs the command and returns "hello"

☐

`go build ./...` compiles ALL packages without errors

☐

`docker-compose up` starts all services, health checks pass

- ☐ make `test` runs unit tests with mocks only where allowed
- ☐ make `integration-test` runs against real PostgreSQL + Redis
- ☐ `./tests/e2e/challenges/run_all_challenges.sh` passes ALL challenges
- ☐ NO “simulated”, “for now”, “TODO”, “placeholder” text in production code
- ☐ CONSTITUTION.md, CLAUDE.md, AGENTS.md exist at root and in all submodules
- ☐ All features advertised in README are exercised by at least one challenge
- ☐ HelixCode exceeds Aider’s capability in: MCP support, distributed workers, memory integration
- ☐ 100% test coverage (unit, integration, E2E, security, benchmark)
- ☐ 100% challenge coverage (every component has challenge)
- ☐ All 15+ LLM providers have real implementations
- ☐ All 40+ tools have real implementations
- ☐ All 9 memory providers have real implementations
- ☐ All 4 notification channels work
- ☐ All submodules have governance files

Appendix A: Reference Implementations to Port from HelixAgent

HelixAgent File	HelixCode Target	Description
<code>internal/llm/circuit_breaker.go</code>	<code>internal/llm/circuit_breaker.go</code>	Production circuit breaker
<code>internal/llm/ensemble.go</code>	<code>internal/llm/ensemble.go</code>	Parallel provider execution
<code>internal/llm/providers/*/provider.go</code>	<code>internal/llm/providers/*/</code>	Real provider implementations
<code>internal/tools/handler.go</code>	<code>internal/tools/</code>	1000+ line tool handler
<code>internal/services/boot_manager.go</code>	<code>internal/server/boot.go</code>	Service boot orchestration

HelixAgent File	HelixCode Target	Description
challenges/scripts/run_all_challenges.sh	tests/e2e/challenges/run_all.sh	Master challenge runner
challenges/scripts/challenge_framework.sh	tests/e2e/challenges/framework.sh	Challenge framework
CONSTITUTION.md	CONSTITUTION.md	36-rule governance
CLAUDE.md	CLAUDE.md	70KB agent manual
docker-compose.yml	docker-compose.yml	Production orchestration
k8s/	k8s/	Kubernetes manifests
internal/skills/	internal/skills/	Skills system

Appendix B: File Creation Checklist

New Files to Create:

- ☐ CONSTITUTION.md (root, 330+ lines)
- ☐ CLAUDE.md (root, 490+ lines)
- ☐ AGENTS.md (root, 390+ lines)
- ☐ helix_code/go.mod (full dependencies)
- ☐ docker/docker-entrypoint.sh
- ☐ internal/llm/provider.go (interface)
- ☐ internal/llm/manager.go (provider manager)
- ☐ internal/llm/circuit_breaker.go
- ☐ internal/llm/providers/ollama/ollama.go
- ☐ internal/llm/providers/openai/openai.go
- ☐ internal/llm/providers/anthropic/anthropic.go
- ☐ internal/llm/providers/gemini/gemini.go
- ☐ internal/llm/providers/xai/xai.go
- ☐ internal/llm/providers/openrouter/openrouter.go

- ☐ internal/llm/providers/github/github.go
- ☐ internal/llm/providers/qwen/qwen.go
- ☐ internal/llm/providers/llamacpp/llamacpp.go
- ☐ internal/llm/providers/vllm/vllm.go
- ☐ internal/llm/providers/azure/azure.go
- ☐ internal/llm/providers/bedrock/bedrock.go
- ☐ internal/llm/providers/vertexai/vertexai.go
- ☐ internal/llm/providers/groq/groq.go
- ☐ internal/llm/providers/koboldai/koboldai.go
- ☐ internal/tools/filesystem/*.go (5+ tools)
- ☐ internal/tools/shell/shell.go
- ☐ internal/tools/git/git.go
- ☐ internal/tools/browser/browser.go
- ☐ internal/tools/web/web.go
- ☐ internal/tools/multiedit/multiedit.go
- ☐ internal/tools/mapping/mapping.go
- ☐ internal/editor/editor.go
- ☐ internal/worker/ssh.go
- ☐ internal/worker/pool.go
- ☐ internal/worker/stats.go
- ☐ internal/task/checkpoint.go
- ☐ internal/task/distributor.go
- ☐ internal/workflow/engine.go
- ☐

- ☐ internal/session/manager.go
- ☐ internal/project/manager.go
- ☐ internal/database/migrations/*.sql (11 tables)
- ☐ internal/mcp/server.go
- ☐ internal/memory/providers/*.go (9 providers)
- ☐ internal/notification/channels/*.go (4 channels)
- ☐ internal/server/health.go
- ☐ tests/anti_bluff_framework.go
- ☐ tests/e2e/challenges/run_all_challenges.sh
- ☐ tests/e2e/challenges/*.sh (100+ challenges)
- ☐ k8s/*.yaml (10+ manifests)
- ☐ docs/DEPLOYMENT.md
- ☐ docs/DEVELOPER_GUIDE.md
- ☐ docs/issues/fixed/BUGFIXES.md
- ☐ docs/issues/BLUFFS.md

Files to Update:

- ☐ cmd/cli/main.go (fix BLUFF-001, BLUFF-002, BLUFF-003)
- ☐ Dockerfile (reference correct entrypoint)
- ☐ AGENTS.md (merge with HELIXCODE_AGENTS.md)
- ☐ README.md (reflect reality)

Appendix C: Verification Commands for Every Phase

```
# Verify no bluffs in code
grep -r "simulated\|for now\|TODO implement\|placeholder"
    internal/ cmd/ applications/
```

```

# Expected: NOTHING

# Verify real LLM calls
curl -X POST http://localhost:8080/api/v1/llm/generate \
  -H "Content-Type: application/json" \
  -d '{"prompt":"What is 2+2?","model":"llama3.2"}'
# Should return actual AI-generated text, NOT "This is a
  simulated response"

# Verify model listing is dynamic
curl http://localhost:8080/api/v1/llm/models
# Should return dynamic list, not just 3 hardcoded models

# Verify command execution is real
./bin/cli --command "echo hello world"
# Should output "hello world", not just "Command completed
  successfully"

# Verify go.mod dependencies
cat helix_code/go.mod | grep -E "gin|pgx|viper|cobra|chromedp|
  tview|fyne"
# Should return all advertised dependencies

# Verify tests use real infrastructure
grep -r "t.Skip" tests/ | grep -v "SKIP-OK"
# Should return NOTHING (no bare skips)

# Verify challenges catch bluffs
./tests/e2e/challenges/run_all_challenges.sh
# ALL must PASS

# Verify Constitution propagated
find . -name "CONSTITUTION.md" | wc -l
# Should be 1 + number of submodules with governance

# Verify no mocks above unit tests
make no-mocks-above-unit
# Should pass

```

Conclusion

This Master Action Plan transforms HelixCode from a project with significant bluff areas into a **fully complete, zero-bluff, production-ready platform** that exceeds all Tier 1 CLI agent capabilities.

Key Principles: 1. **No Bluffs:** Every feature **MUST** actually work 2. **Real Infrastructure:** Non-unit tests **MUST** use real services 3. **Anti-Bluff Verification:** Tests/Challenges **MUST** verify

REAL behavior 4. **100% Coverage:** Every component has tests AND challenges 5. **Governance:** Constitution + CLAUDE.md + AGENTS.md everywhere

Execution Order: - Phase 0: Foundation (Week 1) - CRITICAL - Phase 1: LLM Integration (Weeks 2-3) - CRITICAL - Phase 2: Tools & Editor (Weeks 4-5) - Phase 3: Worker & Distributed (Weeks 6-7) - Phase 4: Workflow & Session (Weeks 8-9) - Phase 5: MCP, Memory, Notifications (Weeks 10-11) - Phase 6: Testing & Challenges (Weeks 12-13) - Phase 7: Documentation & Deployment (Week 14)

Success Metric: When `./tests/e2e/challenges/run_all_challenges.sh` passes ALL 100+ challenges, HelixCode is zero-bluff.

This plan is a living document. Every change to HelixCode MUST update this plan and verify against the anti-bluff criteria defined herein.

Total Estimated Time: 14 weeks **Total Tasks:** 200+ fine-grained tasks **Total Challenges:** 100+ **Total Tests:** 500+ (unit, integration, E2E, security, benchmark)